

Lab Experiment: 2

Title: Arduino with Simple Button Interfacing

Course title: *IOT Laboratory*

Course code: *CSE-460*
4th Year 1st Semester Examination 2025

Date of Submission: 21/02/2026



Submitted to-

Dr. Md. Ezharul Islam, B.SC.(JU), M.ENGG.(JAPAN), PHD(JAPAN)

Professor

Department of Computer Science and Engineering
Jahangirnagar University

&

Samsun Nahar Khandakar

Assistant Professor

Department of Computer Science and Engineering
Jahangirnagar University
Savar, Dhaka-1342

Class Roll	Exam Roll	Name
383	210903	Abdullah Nazmus-Sakib
379	210899	Shah Sultan
391	210911	Md Khaled Mahmud

1. Objectives

The main objectives of this experiment were:

- To learn how to interface a digital input device (pushbutton) with the Arduino Uno.
- To understand the concept of reading digital states (HIGH/LOW) using the `digitalRead()` function.
- To control an output device (LED) based on the input status of a pushbutton.

2. Equipment & Components

Based on the lab sheet, the following components were collected:

- **Microcontroller:** Arduino Uno
- **Actuator:** LED (Red)
- **Sensor/Input:** Pushbutton switch
- **Resistors:**
 - Current limiting resistor for LED (typically 220Ω or 330Ω)
 - Pull-down resistor for Button (labeled $4.7k\Omega$ in Fig. 1)
- **Connecting Wires & Breadboard**

3. Theory

- **Digital Input:** Unlike an LED which is an output device, a button is an input device. The Arduino reads the voltage level at the pin to determine if the button is pressed.
- **Pull-down Resistor:** In this circuit, a $4.7k\Omega$ resistor is used as a "pull-down" resistor. It ensures that when the button is *not* pressed, the input pin reads 0V (LOW). Without it, the pin might float and give random readings.
- **Logic:**
 - **Button Pressed:** 5V flows to the pin $\rightarrow$ Arduino reads **HIGH**.
 - **Button Released:** Pin connects to Ground via resistor $\rightarrow$ Arduino reads **LOW**.

4. Procedure

1. **Component Collection:** We gathered the necessary components listed in the requirements.
2. **Circuit Connection:** We built the circuit on the breadboard according to **Fig. 1**:
 - **LED:** Anode connected to **Pin 12**; Cathode connected to Ground (GND).
 - **Button:** One side connected to 5V; the other side connected to **Pin 2** and to Ground via the $4.7k\Omega$ resistor.
3. **Programming:** We opened the Arduino IDE and navigated to File > Examples > Digital > Button as a reference, then adapted the code to match our pin configuration (Pin 2 for button, Pin 12 for LED).

5. Source Code

The following code was uploaded to the Arduino board:

```
C++
const int buttonPin = 2;    // the number of the pushbutton pin
const int ledPin = 12;      // the number of the LED pin

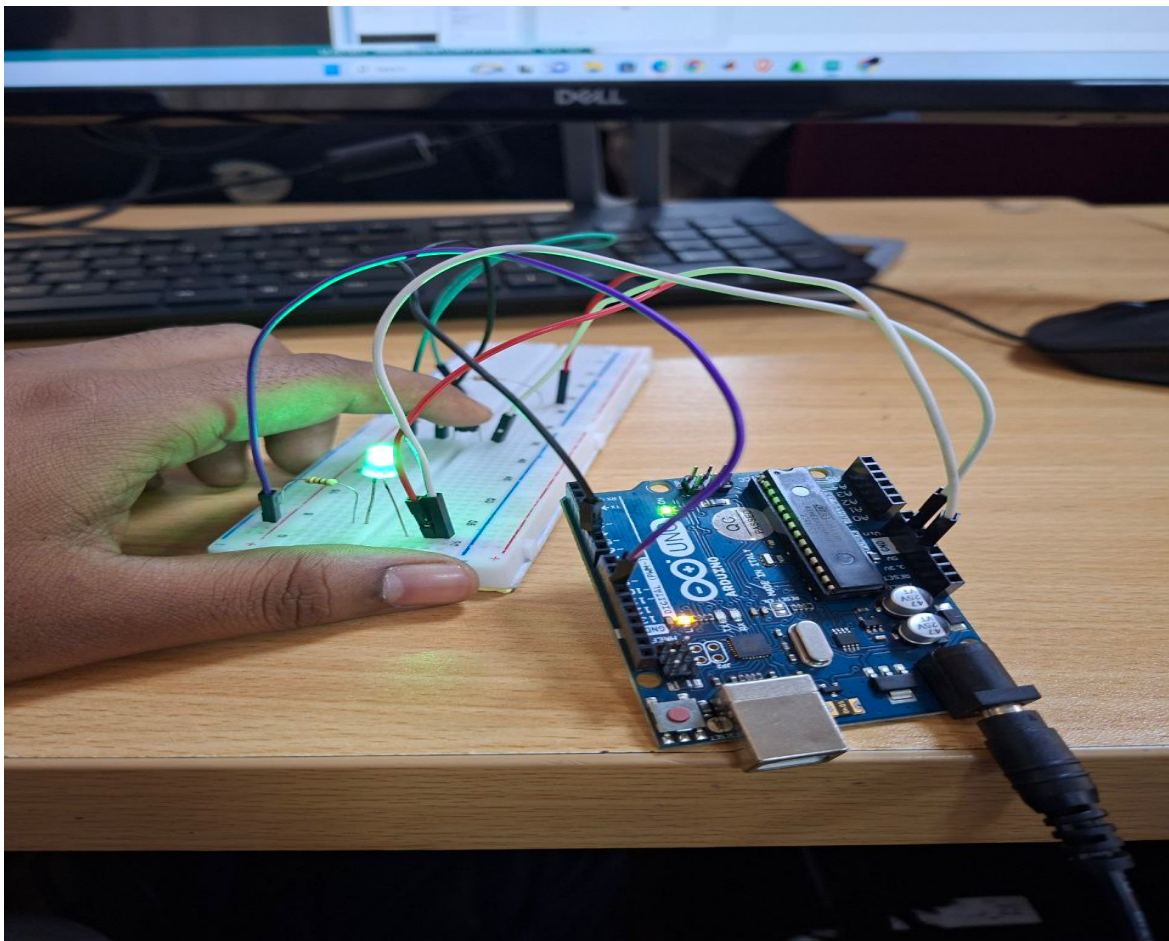
int buttonState = 0;        // variable for reading the pushbutton status

void setup() {
  pinMode(ledPin, OUTPUT);   // initialize the LED pin as an output
  pinMode(buttonPin, INPUT); // initialize the pushbutton pin as an input
}

void loop() {
  // read the state of the pushbutton value:
  buttonState = digitalRead(buttonPin);

  // check if the pushbutton is pressed. if it is, the buttonState is HIGH:
  if (buttonState == HIGH) {
    // turn LED on:
    digitalWrite(ledPin, HIGH);
  } else {
    // turn LED off:
    digitalWrite(ledPin, LOW);
  }
}
```

6. Circuit Analysis (Fig. 1)



The schematic provided in the lab sheet illustrates:

- **Pin 12 (Output):** Sends voltage to the LED when the code instructs it to turn on.
- **Pin 2 (Input):** constantly monitors the voltage.
- **Resistor Configuration:** The 4.7k Ω resistor (R2) is critical. It creates a default state of "LOW" for Pin 2. When the switch closes, the direct connection to 5V overpowers the weak connection to the ground, pulling the pin "HIGH".

7. Observation & Conclusion

Observation:

- When the system was powered on, the LED remained OFF initially.
- When the pushbutton was pressed and held, the LED turned **ON**.
- When the pushbutton was released, the LED immediately turned **OFF**.

Conclusion:

We successfully interfaced a pushbutton with the Arduino. We learned how to configure a GPIO pin as INPUT and use conditional statements (if/else) to control an output based on real-time user input. This experiment demonstrates the fundamental logic required for controlling devices via switches.